

Submarine Simulator

Simple OpenGL / GLUT Function Guide

75 functions • simple explanations • every function in a separate box

How to use: Learn the common OpenGL calls once. Then use each function box to quickly understand what that function does. Repeated calls are not explained again.

Common OpenGL / GLU / GLUT Calls

These are the calls used repeatedly throughout the project.

Layer — Main functions (Role)

Input — keyboardDown / keyboardUp / specialKeys / mouse / motion (Changes state and camera controls)

Update — timer / updateSubmarine / updateIntro / updateCrewWalk (Advances movement, animation and transitions)

Camera — setupCamera / setupFog (Builds the current 3D view and underwater depth effect)

World — drawSky / drawOceanSurface / terrain / marine-life functions (Renders the environment)

Submarine — drawFullSubmarine and its component functions (Renders the vehicle and lights)

Interior — drawInterior / drawCrewCabin (Renders first-person submarine spaces)

UI — drawHUD / drawMenuScreen / drawIntroOverlay (Renders 2D overlays)

glRasterPos2f — Sets the 2D raster position where bitmap text starts. (x,y = 2D raster/text origin.)

glutBitmapCharacter — Draws one bitmap font character at the current raster position. (Font + character code draw one bitmap character.)

glRasterPos3f — Sets a 3D raster position where bitmap text is drawn. (x,y,z = 3D raster/text anchor.)

glutSolidSphere — GLUT helper that draws a solid sphere with a specified radius and tessellation.

gluCylinder — Generates a cylinder around the quadric's local axis. With the second radius set to 0, the same call produces a cone.

gluDeleteQuadric — Releases a GLU quadric object after use.

gluNewQuadric — Creates a GLU quadric object used by GLU shape generators.

gluQuadricNormals — Requests normals for GLU-generated geometry; GLU_SMOOTH gives smooth lighting across the surface.

gluDisk — Generates a flat circular disk/ring between inner and outer radii.

glutSolidCube — GLUT helper that draws a solid cube with a specified side length.

glutSolidTorus — GLUT helper that draws a solid torus using inner/outer radii and tessellation counts.

glBindTexture — Makes a texture object active for the current texture target; subsequent textured geometry samples it. (Target + texture ID select the active texture.)

glGenTextures — Allocates OpenGL texture-object IDs that the program stores in variables such as texSky and texWater. (Count + output array; the returned IDs identify texture objects.)

glTexImage2D — Uploads a 2D pixel array into an OpenGL texture, specifying dimensions, format and data type. (Width/height and RGB/RGBA format describe the uploaded pixel buffer.)

glTexParameterf — Sets texture sampling/wrapping parameters. This project uses it for linear filtering and repeat/clamp behavior. (This source sets linear filtering and repeat/clamp wrapping.)

glBegin — Starts immediate-mode vertex specification for the selected primitive type, such as GL_QUADS or GL_TRIANGLES. (Primitive type controls how vertices are grouped into geometry.)

glEnd — Ends the glBegin/glEnd vertex specification block.

glGetFloatv — Reads a floating-point OpenGL state. drawBillboard reads the model-view matrix to align its quad with the camera. (Reads the requested OpenGL state into a float array.)

glTexCoord2f — Assigns a 2D texture coordinate (u,v) to the next vertex. (u, v = texture coordinates, normally 0–1.)

glVertex3f — Supplies a 3D vertex position to the primitive currently open with glBegin. (x, y, z = vertex position in the current coordinate system.)

glBlendFunc — Defines how source and destination colors are combined when blending is enabled. This creates transparency and soft effects. (Source/destination factors determine how transparency is composited.)

glColor3f — Sets the current RGB drawing color using floating-point values, normally in the 0–1 range.

glColor4f — Sets RGBA color. Alpha is used with blending for transparent/soft effects. (r, g, b, a; alpha controls opacity when blending is active.)

glColorMaterial — Allows vertex colors from glColor calls to control selected material properties while lighting is enabled.

glDisable — Disables an OpenGL capability so it does not affect subsequent drawing.

glEnable — Enables an OpenGL capability, such as lighting, blending, texturing or fog.

glMaterialf — Sets one floating-point material property, such as shininess. (Property/value controls a scalar material setting.)

glMaterialfv — Sets a material property from a float array, such as ambient, diffuse, specular or emission color. (Property plus float array controls material color/response.)

glNormal3f — Supplies a surface normal. With lighting enabled, the normal determines how the surface responds to light. (nx, ny, nz = surface normal direction.)

glPopMatrix — Restores the matrix saved by glPushMatrix, preventing one object's transforms from affecting later objects.

glPushMatrix — Saves the current model-view transformation matrix. It creates a local transform scope for one object or object part.

glRotatef — Rotates subsequent geometry by an angle in degrees around the supplied axis vector. The project uses it for submarine orientation, propeller spin, creature motion and vegetation sway. (angle in degrees; x,y,z define the rotation axis.)

glScalef — Scales subsequent geometry independently along x, y and z. It turns generic primitives into the required shapes. (sx, sy, sz = scale factors along each axis.)

glTranslatef — Moves the coordinate origin by x, y, z. Geometry drawn afterward is positioned relative to that new origin. (x, y, z = displacement of the local origin.)

glLightf — Sets one floating-point property of an OpenGL light, such as spotlight cutoff/exponent or attenuation. (Property/value pair controls one light parameter.)

glLightfv — Sets light properties from float arrays, including position, direction and light colors. (Property plus float array controls vector/color light data.)

glIsEnabled — Checks whether an OpenGL capability is currently enabled, allowing the renderer to restore the previous state correctly. (Capability query returns whether a state is currently enabled.)

gluQuadricTexture — Requests generated texture coordinates for GLU quadric geometry.

gluSphere — Generates a sphere using the GLU quadric.

glLightModelfv — Sets a global lighting-model property, such as scene-wide ambient light.

glLineWidth — Sets the pixel width of lines used by line primitives.

glShadeModel — Selects flat or smooth shading for lighting interpolation across geometry.

glLoadIdentity — Resets the selected matrix to the identity matrix before building a new view/projection or 2D coordinate system.
glMatrixMode — Selects the matrix being changed, such as GL_MODELVIEW or GL_PROJECTION. (Common modes here: GL_MODELVIEW and GL_PROJECTION.)
glVertex2f — Supplies a 2D vertex position, mainly for screen-space HUD/menu rectangles. (x, y = vertex position in the current 2D projection.)
gluOrtho2D — Creates a flat orthographic projection for 2D HUD/menu coordinates without perspective. (Left, right, bottom, top define flat 2D coordinates.)
glFogf — Sets a single floating-point fog parameter, such as density or start/end distance. (Parameter/value controls scalar fog settings.)
glFogfv — Sets fog color or another fog parameter from a float array. (Parameter plus array controls fog color or vector settings.)
glFogi — Sets an integer fog parameter, such as the fog mode. (Integer fog parameter such as fog mode.)
glHint — Gives OpenGL a quality/performance preference for a feature; the project uses it for fog. (Target/mode requests a preferred implementation quality.)
gluLookAt — Builds a camera view from eye position, target position and up direction. (Eye, target, up vector define the view.)
gluPerspective — Creates a perspective projection from field of view, aspect ratio and near/far clipping planes. (Field of view, aspect ratio, near clip, far clip.)
glClear — Clears selected buffers, primarily the color and depth buffers at the beginning of a frame. (Mask selects which buffers are cleared.)
glClearColor — Sets the color used when the color buffer is cleared.
glutSwapBuffers — Swaps the back and front buffers in double-buffered rendering so the completed frame becomes visible.
glutPostRedisplay — Marks the window for redraw, causing the display callback to run again.
glViewport — Defines the pixel rectangle of the window where OpenGL renders. (x, y, width, height define the render rectangle.)
glutTimerFunc — Schedules a timer callback after a specified interval. (Delay + callback + user value schedule the next update.)
glutCreateWindow — Creates the GLUT window and OpenGL context.
glutDisplayFunc — Registers the display callback used to render frames.
glutInit — Initializes the GLUT library.
glutInitDisplayMode — Chooses the window/display buffers and modes, such as RGB, double buffering and depth buffering.
glutInitWindowPosition — Sets the initial window position on the desktop.
glutInitWindowSize — Sets the initial window width and height.
glutKeyboardFunc — Registers the normal keyboard press callback.
glutKeyboardUpFunc — Registers the normal keyboard release callback.
glutMainLoop — Used by the source; see the function section for its project-specific role.
glutMotionFunc — Registers the mouse-drag callback.
glutMouseFunc — Registers the mouse-button callback.
glutReshapeFunc — Registers the callback for window-size changes.
glutSpecialFunc — Registers the special-key callback for keys such as arrows/function keys.
Question — Answer

Why use `glPushMatrix/glPopMatrix`? — To isolate an object's transformations so moving or rotating one part does not permanently affect other objects.

Why use `glScalef` after a primitive? — The primitive is generic; scaling changes its dimensions into the required project-specific shape.

Why use `glNormal3f`? — Lighting needs a surface direction to calculate how strongly light should affect a surface.

Why use `glColor4f` instead of `glColor3f`? — The fourth value is alpha, which supports transparent effects when blending is enabled.

Why call `glBlendFunc`? — Blending defines how the new source color is combined with the already-rendered destination color.

What is `glBindTexture` doing? — It selects the texture object that the following textured geometry will use.

Why `glTexCoord2f`? — It tells OpenGL which point in the 2D texture image belongs to a vertex.

Why `gluPerspective`? — It creates perspective projection so distant objects appear smaller and the scene has depth.

Why `gluOrtho2D` for the HUD? — The HUD needs flat screen coordinates rather than perspective distortion.

What does `glMatrixMode` do? — It selects whether subsequent matrix operations affect the projection matrix or model-view matrix.

What does `glLoadIdentity` do? — It clears the previous transformation in the selected matrix so a new camera/projection setup can be built cleanly.

What is the difference between `glBegin/glEnd` and GLUT primitives? — `glBegin/glEnd` lets the project define custom vertices; GLUT primitives provide ready-made solid shapes such as spheres, cubes and tori.

Why use GLU quadrics? — They provide convenient generated geometry such as cylinders, disks and spheres, with optional normals/textures.

Why use fog underwater? — It reduces distant visibility and creates a depth cue that makes the underwater environment feel less flat.

What is the role of `display()`? — It is the per-frame renderer that turns the current simulation state into the visible frame.

What is the role of `timer()`? — It advances the simulation/update cycle and requests continued redraws.

Why are GLUT callback functions important? — GLUT is event-driven; keyboard, mouse, resize and timer events call the registered functions.

1. Reusable Drawing Primitives & Text Helpers

drawText

Source: lines 234–239. Places a 2D bitmap string at the supplied screen-space position by setting a raster position and sending each character to GLUT.

What it does: `glRasterPos2f(x, y)`

Important calls: `glRasterPos2f`: Sets the 2D raster position where bitmap text starts.; `glutBitmapCharacter`: Draws one bitmap font character at the current raster position.

Example: `glRasterPos2f(x, y) | glutBitmapCharacter(font, *c)`

drawText3D

Source: lines 240–246. Places bitmap text at a 3D world coordinate, so text can be anchored to scene geometry.

What it does: `glRasterPos3f(x, y, z)`

Important calls: `glRasterPos3f`: Sets a 3D raster position where bitmap text is drawn.

Example: `glRasterPos3f(x, y, z)`

drawSphere

Source: lines 251–253. Reusable wrapper around GLUT's solid sphere primitive.

What it does: `glutSolidSphere(radius, slices, stacks)`

Important calls: `glutSolidSphere`: GLUT helper that draws a solid sphere with a specified radius and tessellation.

Example: `glutSolidSphere(radius, slices, stacks)`

drawCylinder

Source: lines 254–260. Creates a GLU quadric, requests smooth normals, generates a cylinder, then releases the quadric object.

What it does: `gluCylinder(quad, radius, radius, height, slices, 1)`

Important calls: `gluCylinder`: Generates a cylinder around the quadric's local axis. With the second radius set to 0, the same call produces a cone.; `gluDeleteQuadric`: Releases a GLU quadric object after use.; `gluNewQuadric`: Creates a GLU quadric object used by GLU shape generators.; `gluQuadricNormals`: Requests normals for GLU-generated geometry; `GLU_SMOOTH` gives smooth lighting across the surface.

Example: `gluCylinder(quad, radius, radius, height, slices, 1) | gluDeleteQuadric(quad) | gluNewQuadric() | gluQuadricNormals(quad, GLU_SMOOTH)`

drawCone

Source: lines 261–267. Uses a GLU cylinder with a zero top radius, making the generated shape a cone.

Also uses: `gluCylinder`, `gluDeleteQuadric`, `gluNewQuadric`, `gluQuadricNormals`

drawDisk

Source: lines 268–273. Generates a flat circular disk/ring between the supplied inner and outer radii.

What it does: `gluDisk(quad, innerRadius, outerRadius, slices, 1)`

Important calls: `gluDisk`: Generates a flat circular disk/ring between inner and outer radii.

Example: `gluDisk(quad, innerRadius, outerRadius, slices, 1)`

drawCube

Source: lines 274–277. Reusable wrapper for a solid GLUT cube.

What it does: `glutSolidCube(size)`

Important calls: `glutSolidCube`: GLUT helper that draws a solid cube with a specified side length.

Example: `glutSolidCube(size)`

drawTorus

Source: lines 278–281. Reusable wrapper for a solid torus/ring.

What it does: `glutSolidTorus(inner, outer, sides, rings)`

Important calls: `glutSolidTorus`: GLUT helper that draws a solid torus using inner/outer radii and tessellation counts.

Example: `glutSolidTorus(inner, outer, sides, rings)`

drawBillboard

Source: lines 506–519. Builds a textured quad whose horizontal and vertical axes are taken from the current model-view matrix, so the quad faces the camera.

What it does: `glBegin(GL_QUADS)`

Important calls: `glBegin`: Starts immediate-mode vertex specification for the selected primitive type, such as `GL_QUADS` or `GL_TRIANGLES`.; `glEnd`: Ends the `glBegin/glEnd` vertex specification block.; `glGetFloatv`: Reads a floating-point OpenGL state. `drawBillboard` reads the model-view matrix to align its quad with the camera.; `glTexCoord2f`: Assigns a 2D texture coordinate (u,v) to the next vertex.; `glVertex3f`: Supplies a 3D vertex position to the primitive currently open with `glBegin`.

Example: `glBegin(GL_QUADS) | glEnd() | glGetFloatv(GL_MODELVIEW_MATRIX, m) | glTexCoord2f(0, 0)`

2. Procedural Noise & Math Helpers

hashNoise

Source: lines 288–293. Produces a deterministic pseudo-random integer from x, y and a seed. It is a math helper, not a rendering call.

noise01

Source: lines 294–296. Converts the hash value to a normalized pseudo-random floating-point value used by procedural textures and terrain.

smoothNoise

Source: lines 297–304. Samples neighboring noise values and interpolates them smoothly, reducing abrupt changes in procedural patterns.

seaFloorY

Source: lines 307–313. Computes the seabed height from several sine/cosine layers, producing rolling underwater terrain.

What it does: Functions that prepare the scene before normal frame rendering starts.

3. Setup & Initialization

uploadTexture

Source: lines 314–324. Creates an OpenGL texture object, sets filtering/wrapping parameters, uploads the supplied RGB/RGBA pixel data, and returns the texture ID.

What it does: `glBindTexture(GL_TEXTURE_2D, t)`

Important calls: `glBindTexture`: Makes a texture object active for the current texture target; subsequent textured geometry samples it.; `glGenTextures`: Allocates OpenGL texture-object IDs that the program stores in variables such as `texSky` and `texWater`.; `glTexImage2D`: Uploads a 2D pixel array into an OpenGL texture, specifying dimensions, format and data type.; `glTexParameterf`: Sets texture sampling/wrapping parameters. This project uses it for linear filtering and repeat/clamp behavior.

Example: `glBindTexture(GL_TEXTURE_2D, t) | glGenTextures(1, &t) | glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR) |` What to remember: Texture creation is a two-stage idea: create/select the texture object, then upload pixels and define how OpenGL samples them.

initTextures

Source: lines 325–503. Generates procedural textures for the sky, water, sand, hull and soft sprites, and attempts to load the two BMP-based textures used by the project.

initSubmarine

Source: lines 524–532. Initializes the submarine's position, orientation, speed, propeller angle, headlight state, depth and descent rate.

initCamera

Source: lines 533–540. Sets the starting orbit-camera angles/distance and the free-camera position and orientation.

initEnvironment

Source: lines 541–761. Creates the initial populations and placement data for fish, seaweed, rocks, coral, sharks, rays, kelp, crew, particles, clouds and trees.

setupLighting

Source: lines 4799–4849. Enables and configures the lighting system, light sources, global ambient light, material response and smooth shading.

What it does: What to remember: Lighting only affects the scene when the required lighting state is enabled and geometry has useful normals/material properties.

Also uses: `glColorMaterial`, `glDisable`, `glEnable`, `glLightModelfv`, `glLightf`, `glLightfv`, `glMaterialf`, `glMaterialfv`, `glShadeModel`

setupFog

Source: lines 4850–4877. Configures underwater depth fog, including its mode, color and distance/density parameters.

What it does: `glFogf(GL_FOG_DENSITY, fogDensity)`

Important calls: `glFogf`: Sets a single floating-point fog parameter, such as density or start/end distance.; `glFogfv`: Sets fog color or another fog parameter from a float array.; `glFogi`: Sets an integer fog parameter, such as the fog mode.; `glHint`: Gives OpenGL a quality/performance preference for a feature; the project uses it for fog.

Example: `glFogf(GL_FOG_DENSITY, fogDensity) | glFogfv(GL_FOG_COLOR, fogColor) | glFogi(GL_FOG_MODE, GL_EXP2) | glHint(GL_FOG_HINT, GL_DONT_CARE)`

setupCamera

Source: lines 4882–5000. Builds the current camera projection and view. Perspective is used for 3D views and `gluLookAt` positions the viewer for the active camera mode.

What it does: `gluLookAt(camX, camY, camZ, lookX, lookY, sub.z, 0, 1, 0)`

Important calls: `gluLookAt`: Builds a camera view from eye position, target position and up direction.; `gluPerspective`: Creates a perspective projection from field of view, aspect ratio and near/far clipping planes.

Example: `gluLookAt(camX, camY, camZ, lookX, lookY, sub.z, 0, 1, 0) | gluPerspective(fovY, (double>windowWidth / windowHeight, 0.1, 500.0) |`
What to remember: Projection answers “how is 3D mapped to the screen?”, while the model-view transform answers “where is the camera and scene?”. | Functions responsible for the submarine model, its moving propeller and its visible/actual headlight system.

4. Submarine Body, Propeller & Headlights

drawSubmarineBody

Source: lines 766–1172. Constructs the submarine hull, sail, fins and hatch geometry from transformed primitives and custom vertices, with normals/materials used for lighting.

What it does: `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`

Important calls: `glBlendFunc`: Defines how source and destination colors are combined when blending is enabled. This creates transparency and soft effects.; `glColor3f`: Sets the current RGB drawing color using floating-point values, normally in the 0–1 range.; `glColor4f`: Sets RGBA color. Alpha is used with blending for transparent/soft effects.; `glColorMaterial`: Allows vertex colors from `glColor` calls to control selected material properties while lighting is enabled.; `glDisable`: Disables an OpenGL capability so it does not affect subsequent drawing.; `glEnable`: Enables an OpenGL capability, such as lighting, blending, texturing or fog.; `glMaterialf`: Sets one floating-point material property, such as shininess.; `glMaterialfv`: Sets a material property from a float array, such as ambient, diffuse, specular or emission color.; `glNormal3f`: Supplies a surface normal. With lighting enabled, the normal determines how the surface responds to light.; `glPopMatrix`: Restores the matrix saved by `glPushMatrix`, preventing one object's transforms from affecting later objects.; `glPushMatrix`: Saves the current model-view transformation matrix. It creates a local transform scope for one object or object part.; `glRotatef`: Rotates subsequent geometry by an angle in degrees around the supplied axis vector. The project uses it for submarine orientation, propeller spin, creature motion and vegetation sway.; `glScalef`: Scales subsequent geometry independently along x, y and z. It turns generic primitives into the required shapes.; `glTranslatef`: Moves the coordinate origin by x, y, z. Geometry drawn afterward is positioned relative to that new origin.

Example: `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA) | glColor3f(cA[0], cA[1], cA[2]) | glColor4f(0.32f, 0.64f, 0.90f, 0.42f) | glColorMaterial(GL_FRONT_AND_BACK, GL_DIFFUSE)`

drawPropeller

Source: lines 1173–1207. Builds the rear propeller and rotates it according to the submarine's current motion.

Also uses: `glColor3f`, `glPopMatrix`, `glPushMatrix`, `glRotatef`, `glScalef`, `glTranslatef`

setupSubHeadlight

Source: lines 1208–1281. Configures the submarine headlight as an OpenGL light, including its position, direction and spotlight parameters.

What it does: `glLightf(GL_LIGHT2, GL_SPOT_CUTOFF, 28.0f)`

Important calls: `glLightf`: Sets one floating-point property of an OpenGL light, such as spotlight cutoff/exponent or attenuation.; `glLightfv`: Sets light properties from float arrays, including position, direction and light colors.

Example: `glLightf(GL_LIGHT2, GL_SPOT_CUTOFF, 28.0f) | glLightfv(GL_LIGHT2, GL_POSITION, pos2)`

drawHeadlightBeam

Source: lines 1282–1292. Positions the visible geometric beam in front of the submarine and creates the cone-shaped volume.

Also uses: glPopMatrix, glPushMatrix, glRotatef, glTranslatef, gluCylinder, gluDeleteQuadric, gluNewQuadric, gluQuadricNormals

drawBeamCone

Source: lines 1293–1321. Builds the translucent cone mesh used to visually represent the headlight beam.

Also uses: glBegin, glColor4f, glEnd, glPopMatrix, glPushMatrix, glTranslatef, glVertex3f

drawSubmarineHeadlights

Source: lines 1322–1334. Checks the headlight state, configures blending for the visible beam, and draws the beam when appropriate.

Also uses: glBlendFunc, glDisable, glEnable

drawFullSubmarine

Source: lines 1335–1395. Top-level submarine renderer. It applies the submarine's world transform and calls the body, propeller and headlight components.

What it does: glIsEnabled(GL_FOG)

Important calls: glIsEnabled: Checks whether an OpenGL capability is currently enabled, allowing the renderer to restore the previous state correctly.

Example: glIsEnabled(GL_FOG)

updateSubmarine

Source: lines 5005–5145. Updates submarine movement and orientation from controls and dive state. It changes simulation state rather than drawing geometry.

What it does: Surface-level world elements shown before and around the dive transition.

5. Sky, Ocean Surface & Above-Water Environment

drawSky

Source: lines 1400–1435. Renders the sky backdrop using a textured spherical/dome-like surface and controlled orientation.

What it does: gluQuadricTexture(skyQ, GL_TRUE)

Important calls: gluQuadricTexture: Requests generated texture coordinates for GLU quadric geometry.; gluSphere: Generates a sphere using the GLU quadric.

Example: gluQuadricTexture(skyQ, GL_TRUE) | gluSphere(skyQ, 230, 28, 18)

drawSun

Source: lines 1436–1498. Draws a glowing sun element using a textured/translucent quad.

Also uses: glBegin, glBindTexture, glBlendFunc, glColor3f, glColor4f, glDisable, glEnable, glEnd, glPopMatrix, glPushMatrix, glTranslatef, glVertex3f

drawDistantHills

Source: lines 1499–1595. Creates simple horizon silhouettes from custom vertices.

Also uses: glBegin, glColor3f, glDisable, glEnable, glEnd, glPopMatrix, glPushMatrix, glVertex3f

drawBowFoam

Source: lines 1596–1628. Adds semi-transparent foam around the submarine bow near the surface.

Also uses: glBlendFunc, glColor4f, glDisable, glEnable, glPopMatrix, glPushMatrix, glScalef, glTranslatef

drawDeckCrewSilhouettes

Source: lines 1629–1663. Places simplified human silhouettes on the submarine deck using transformed primitives.

Also uses: glColor3f, glDisable, glEnable, glPopMatrix, glPushMatrix, glRotatef, glScalef, glTranslatef

drawClouds

Source: lines 1664–1708. Draws cloud sprites using the generated soft-puff texture with alpha blending.

Also uses: glBindTexture, glBlendFunc, glColor4f, glDisable, glEnable

drawBirds

Source: lines 1709–1765. Draws simple animated bird shapes above the water and changes their orientation as they move.

Also uses: glBegin, glBlendFunc, glColor4f, glDisable, glEnable, glEnd, glPopMatrix, glPushMatrix, glRotatef, glTranslatef, glVertex3f

drawOceanSurface

Source: lines 1766–1878. Builds the water surface from vertices, texture coordinates and normals, with animated/noise-based height variation and transparency.

What it does: What to remember: The surface combines geometry, texture coordinates, normals and blending. Vertex positions provide the shape; texture coordinates provide surface detail; normals make lighting respond to the surface.

Also uses: glBegin, glBindTexture, glBlendFunc, glColor3f, glColor4f, glDisable, glEnable, glEnd, glNormal3f, glPopMatrix, glPushMatrix, glTexCoord2f, glVertex3f

drawTrees

Source: lines 1879–1954. Places simple procedural trees on nearby land/hills using transformed primitive geometry.

What it does: Seabed geometry and underwater plants.

Also uses: glColor3f, glDisable, glEnable, glPopMatrix, glPushMatrix, glRotatef, glScalef, glTranslatef

6. Underwater Terrain & Flora

drawSeaweed

Source: lines 1955–1973. Draws seabed seaweed strands using the generated position, height and phase data.

Also uses: glColor3f, glPopMatrix, glPushMatrix, glScalef, glTranslatef

drawRocks

Source: lines 1974–1984. Draws scattered seabed rocks by translating and scaling primitive geometry for each rock record.

Also uses: glColor3f, glPopMatrix, glPushMatrix, glScalef, glTranslatef

drawCoral

Source: lines 1985–2018. Draws coral clusters with per-instance position, scale, color, type and orientation.

Also uses: glColor3f, glPopMatrix, glPushMatrix, glRotatef, glScalef, glTranslatef

drawKelp

Source: lines 2537–2597. Draws taller kelp strands and applies phase-based rotation to suggest swaying.

What it does: Animated animals, bubbles, particles and underwater light effects.

Also uses: glColor3f, glPopMatrix, glPushMatrix, glRotatef, glScalef, glTranslatef

7. Marine Life & Water Effects

drawFish

Source: lines 2163–2374. Constructs and animates fish/jellyfish geometry using custom vertices, transforms, normals, materials, color and blending.

What it does: What to remember: The fish renderer combines reusable transforms with immediate-mode custom vertices. Animation changes transform/geometry values over time rather than using a separate animation API.

Also uses: glBegin, glBlendFunc, glColor3f, glColor4f, glDisable, glEnable, glEnd, glMaterialfv, glNormal3f, glPopMatrix, glPushMatrix, glRotatef, glScalef, glTranslatef, glVertex3f

drawSharks

Source: lines 2375–2509. Constructs shark-like geometry and moves each shark around its circular patrol path.

Also uses: glBegin, glColor3f, glEnd, glMaterialfv, glNormal3f, glPopMatrix, glPushMatrix, glRotatef, glScalef, glTranslatef, glVertex3f

drawRays

Source: lines 2510–2536. Draws manta-ray-like creatures and animates their position/orientation and fin movement.

Also uses: glColor3f, glPopMatrix, glPushMatrix, glRotatef, glScalef, glTranslatef

drawBubbles

Source: lines 2598–2620. Draws rising translucent bubble shapes from the bubble state array.

Also uses: glBlendFunc, glColor4f, glDisable, glEnable, glPopMatrix, glPushMatrix, glTranslatef

drawParticles

Source: lines 2621–2639. Draws floating debris/plankton particles as small translucent spheres.

Also uses: glBlendFunc, glColor4f, glDisable, glEnable, glPopMatrix, glPushMatrix, glTranslatef, glutSolidSphere

drawLightRays

Source: lines 2640–2688. Creates translucent geometric shafts that simulate underwater light rays.

What it does: Math-only helpers that keep the free camera and interior navigation from clipping through the submarine hull.

Also uses: glBegin, glBlendFunc, glColor4f, glDisable, glEnable, glEnd, glVertex3f

8. Hull Collision & Coordinate-Space Helpers

subWorldToHullLocal

Source: lines 2030–2052. Transforms a world-space point into submarine-local coordinates by reversing the submarine's translation, rotation and scale.

subHullLocalToWorld

Source: lines 2053–2070. Transforms a submarine-local point into world coordinates by applying the submarine's scale, rotation and translation.

inSuperstructureLocal

Source: lines 2075–2083. Tests whether a local-space point lies inside the defined sail/superstructure bounds.

inHullLocal

Source: lines 2087–2102. Tests whether a local-space point lies inside the solid hull or superstructure volume.

resolveHullLocal

Source: lines 2107–2143. If a local point is inside the hull, moves it outward to a valid location with a clearance margin.

insideCrewCabin

Source: lines 2147–2151. Checks whether a world-space point is inside the walkable interior/cabin region.

resolveSubHullBoundary

Source: lines 2155–2162. Wraps hull collision resolution in world space so the camera or another point cannot remain inside solid hull geometry.

cabinNearHatch

Source: lines 95–98. Checks whether the free camera is close enough to the crew-cabin hatch for the hatch interaction.

What it does: The detailed interior environment, including the control room and crew living area.

9. Submarine Interior & Crew Cabin

drawInterior

Source: lines 2693–2968. Renders the submarine control-room interior, including structural panels, consoles and windows, using transformed primitives and lighting/material states.

Also uses: `glBlendFunc`, `glColor3f`, `glColor4f`, `glColorMaterial`, `glDisable`, `glEnable`, `glMaterialfv`, `glPopMatrix`, `glPushMatrix`, `glRotatef`, `glScalef`, `glTranslatef`

drawCrewCabin

Source: lines 2973–4658. The largest interior renderer. It builds the crew living area, bunks, furniture, lighting fixtures, detailed surfaces and crew-related geometry.

What it does: `glLightModelfv(GL_LIGHT_MODEL_AMBIENT, cabAmbient)`

Important calls: `glLightModelfv`: Sets a global lighting-model property, such as scene-wide ambient light.; `glLineWidth`: Sets the pixel width of lines used by line primitives.; `glShadeModel`: Selects flat or smooth shading for lighting interpolation across geometry.

Example: `glLightModelfv(GL_LIGHT_MODEL_AMBIENT, cabAmbient) | glLineWidth(1.6f) | glShadeModel(GL_SMOOTH) |` What to remember: This function is large because many small interior objects are assembled from the same primitives, transforms, materials, textures and light states.

10. HUD & 2D Screen Overlays

drawHUD

Source: lines 4663–4791. Switches to an orthographic screen-space projection and draws the in-game HUD, including status text, mission information and indicators.

What it does: `glLoadIdentity()`

Important calls: `glLoadIdentity`: Resets the selected matrix to the identity matrix before building a new view/projection or 2D coordinate system.; `glMatrixMode`: Selects the matrix being changed, such as `GL_MODELVIEW` or `GL_PROJECTION`.; `glVertex2f`: Supplies a 2D vertex position, mainly for screen-space HUD/menu rectangles.; `gluOrtho2D`: Creates a flat orthographic projection for 2D HUD/menu coordinates without perspective.

Example: `glLoadIdentity() | glMatrixMode(GL_PROJECTION) | glVertex2f(60, windowHeight - 282) | gluOrtho2D(0, windowWidth, 0, windowHeight)`

drawViewportBorder

Source: lines 4792–4794. Draws a thin 2D rectangular border around the current viewport.

drawMenuScreen

Source: lines 5317–5400. Creates the menu UI in orthographic coordinates and draws selectable menu elements.

Also uses: `glBegin`, `glColor3f`, `glColor4f`, `glDisable`, `glEnable`, `glEnd`, `glLoadIdentity`, `glMatrixMode`, `glPopMatrix`, `glPushMatrix`, `glVertex2f`, `gluOrtho2D`

drawIntroOverlay

Source: lines 5401–5477. Creates the introductory overlay/fade and its skip control in screen space.

What it does: Non-rendering state updates that advance timers and movement.

Also uses: glBegin, glBlendFunc, glColor3f, glColor4f, glDisable, glEnable, glEnd, glLoadIdentity, glMatrixMode, glPopMatrix, glPushMatrix, glVertex2f, gluOrtho2D

11. Per-Frame Update & Animation Logic

updateIntro

Source: lines 5146–5203. Advances intro timing and the surface-to-underwater transition state.

updateCrewWalk

Source: lines 5208–5312. Updates first-person cabin walking position and movement animation while respecting cabin/hull boundaries.

What it does: GLUT callback functions that turn keyboard and mouse input into simulator state changes.

12. Input Handling (Keyboard & Mouse)

keyboardDown

Source: lines 5587–5730. Processes normal key presses and changes movement, camera, mission, headlight and menu/game states.

keyboardUp

Source: lines 5731–5744. Processes key releases so held movement flags can be cleared.

specialKeys

Source: lines 5745–5779. Processes GLUT special-key input such as arrows/function keys and requests redraws when state changes.

What it does: glutPostRedisplay()

Important calls: glutPostRedisplay: Marks the window for redraw, causing the display callback to run again.

Example: glutPostRedisplay()

cabinButtonHit

Source: lines 5792–5832. Projects the cabin button toward screen space using the current camera and checks whether a mouse click hits its area.

introSkipButtonHit

Source: lines 5836–5840. Checks whether a mouse click falls inside the screen-space rectangle used by the intro skip button.

mouse

Source: lines 5841–5887. Handles mouse-button events and routes clicks to menu, intro-skip and cabin-button interactions.

Also uses: glutPostRedisplay

motion

Source: lines 5888–5929. Updates free-camera yaw/pitch while the mouse is dragged.

What it does: The framework glue: frame rendering, resizing, timer scheduling and program startup.

Also uses: glutPostRedisplay

13. Core Render Loop, Window & Program Entry

display

Source: lines 5482–5579. The main per-frame renderer. It clears buffers, configures frame state/fog, sets the scene transform, draws the appropriate world/state content, draws overlays, and swaps buffers.

What it does: `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`

Important calls: `glClear`: Clears selected buffers, primarily the color and depth buffers at the beginning of a frame.; `glClearColor`: Sets the color used when the color buffer is cleared.; `glutSwapBuffers`: Swaps the back and front buffers in double-buffered rendering so the completed frame becomes visible.

Example: `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT) | glClearColor(0.52f * (1 - db) + 0.06f * db * (1 - deepK) | glutSwapBuffers()` | What to remember: This is the central connection between simulation state and what appears on screen. The update callbacks change values; `display` reads those values and renders them.

reshape

Source: lines 5934–5940. Updates the OpenGL viewport and projection when the window size changes.

What it does: `glViewport(0, 0, w, h)`

Important calls: `glViewport`: Defines the pixel rectangle of the window where OpenGL renders.

Example: `glViewport(0, 0, w, h)`

timer

Source: lines 5945–5955. Advances the simulation/update cycle, requests redraws, and schedules the next timer callback.

What it does: `glutTimerFunc(16, timer, 0)`

Important calls: `glutTimerFunc`: Schedules a timer callback after a specified interval.

Example: `glutTimerFunc(16, timer, 0)`

main

Source: lines 5960–6004. Creates the GLUT window/context, initializes the project and rendering systems, registers callbacks, and enters the GLUT main loop.

What it does: `glutCreateWindow("3D Submarine Simulator - OpenGL")`

Important calls: `glutCreateWindow`: Creates the GLUT window and OpenGL context.; `glutDisplayFunc`: Registers the display callback used to render frames.; `glutInit`: Initializes the GLUT library.; `glutInitDisplayMode`: Chooses the window/display buffers and modes, such as RGB, double buffering and depth buffering.; `glutInitWindowPosition`: Sets the initial window position on the desktop.; `glutInitWindowSize`: Sets the initial window width and height.; `glutKeyboardFunc`: Registers the normal keyboard press callback.; `glutKeyboardUpFunc`: Registers the normal keyboard release callback.; `glutMainLoop`: See the central call reference.; `glutMotionFunc`: Registers the mouse-drag callback.; `glutMouseFunc`: Registers the mouse-button callback.; `glutReshapeFunc`: Registers the callback for window-size changes.; `glutSpecialFunc`: Registers the special-key callback for keys such as arrows/function keys.

Example: `glutCreateWindow("3D Submarine Simulator - OpenGL") | glutDisplayFunc(display) | glutInit(&argc, argv) | glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH)`

Quick Viva Revision

What does `glPushMatrix/glPopMatrix` do?

Saves and restores the current transformation, so one object's movement does not affect the next object.

What does `glTranslatef` do?

Moves the object in x, y and z.

What does `glRotatef` do?

Rotates the object around the selected axis.

What does `glScalef` do?

Changes the object's size.

What does `glBegin/glEnd` do?

Starts and ends immediate-mode vertex drawing.

What does glColor3f/glColor4f do?

Sets the drawing color; glColor4f also includes alpha.

What does glEnable/glDisable do?

Turns an OpenGL feature on or off.

What does gluLookAt do?

Builds the camera view from position, target and up direction.

What does glutPostRedisplay do?

Requests another frame to be drawn.

What does glutTimerFunc do?

Schedules a timer callback, used here for animation/update timing.